Week 6
1. UNIX Process Control (II)

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Monday, October 6th, 2025

COMP 2560 System Programming: Process Control II

School of Computer Science
University of Windsor

Instructor: Dr. Andreas Maniatis

Content

Part I
Terminating a Process: `exit()`

Process termination: `exit()`

Synopsis: **`void exit(int status);`**

This call terminates a process and never returns The `status` value is available to the parent process through the `wait()` system call.

When invoked by a process, the `exit()` system call:

`closes` all the process's file descriptors. It `flushes` all output streams and closes all open streams,
`frees` the memory used by its code, data and stack,
`sends` a `SIGCHLD` signal to its parent and `waits` for the parent to accept its return code.

Example (terminate.c)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#include <sys/types.h>
#include <sys/wait.h>

int main() {
  int newpid;
  printf("before: mypid is %d\n", getpid());
  if ((newpid = fork()) == -1 )
        perror("fork");
  else if (newpid == 0){
          printf("I am the child %d now sleeping...\n",getpid());
          sleep(2);
          exit(47);
          printf("I am gone");
        }
        else{
          printf("I am the parent %d\n",getpid());
          sleep(10);
          printf("My child %d must be gone by now. I am leaving...\n",newpid);
          exit(1);
          printf("I am gone too\n");
        }
}
```

What is the output produced?

The output of the previous program is:

```
before:  mypid is 5067
I am the parent 5067
I am the child 5068 now sleeping...
My child 5068 must be gone by now.   I am
leaving...
```

The lines `printf("I am gone");` and `printf("I am gone too");` are not executed because `exit()` never returns.

Part II
Waiting for a Process: `wait()` and `waitpid()`

wait() and waitpid()

wait()

Synopsis: `pid_t wait(int *status);`
Allows the parent to wait for the termination of one of its children and to accept its termination code.

waitpid()

Synopsis: `pid_t waitpid(pid_t pid, int *status, int options);`
`waitpid()` is similar to `wait()` except that it has a number of options that control which process it waits for.
For example:

- If `pid` > 0 then, `waitpid()` waits for the child process identified by `pid`.
- If `pid` = -1 then, `waitpid()` waits for any child process to return.

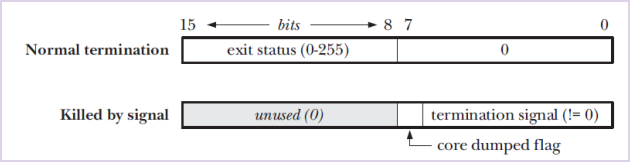
Termination

When successful, `wait(…)` returns the `pid` of the terminating child process.
The value in `status` is encoded as follow:

- if the rightmost byte of `status` is zero, then the leftmost byte contains the status returned by the child: a value between 0 and 255 (passed as an argument to exit).
  This represents a normal termination of the child process.

- if the rightmost byte of `status` is nonzero, then the rightmost 7 bits are equal to the `signal number`, that caused the process to terminate. The remaining bit of the rightmost byte is set to 1 if a core dump was produced by the child process.

Termination



Value returned in the *status* argument of *wait()* and *waitpid()*

Example (terminate2.c)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#include <sys/types.h>
#include <sys/wait.h>

int main() {
  int newpid;
  printf("before: mypid is %d\n", getpid());
  if ((newpid = fork()) == -1 )
        perror("fork");
  else if (newpid == 0){
          printf("I am the child %d now sleeping...\n",getpid());
          sleep(5);
          exit(47);
       }
       else{
          printf("I am the parent %d\n",getpid());
          int status;
          int child_pid  = wait(&status);
          printf("My child %d has terminated\n",child_pid);

          printf("I have received the status = %d\n",status);

          int child_status = status >> 8;
          int signal = status & 0x7F;
          int core = status & 0x80;

          printf("Child status = %d Signal = %d Core = %d\n",child_status, signal,
          core);
       }
}
```

Example of a normal termination

Example

Consider the previous program. The output resulting from a normal execution:

```
before:  mypid is 4602
I am the parent 4602
I am the child 4603 now sleeping...
My child 4603 has terminated
I have received the status = 12032
Child status = 47 Signal = 0 Core = 0
```

Example of a premature termination

Example

1. Consider the previous program running. When the child is sleeping, the output is as follows:

```
before:  mypid is 4602
I am the parent 4602
I am the child 4603 now sleeping...
```

2. Before the child makes the `exit` call, we `kill` the child process. At the command prompt, we run:

```
kill 4603                                    (kill -l)
```

3. The parent wakes up and displays:

```
My child 4603 has terminated
I have received the status = 15  Child
status = 0 Signal = 15 Core =  0
```

Part III
Bit Manipulation Macros

Bit-manipulation macros

Some bit-manipulation macros have been defined to deal with the value in the variable `status` (you need to include `<sys/wait.h>`).

`WIFEXITED(status)`: true for normal child termination.

`WEXITSTATUS(status)`: used only when `WIFEXITED(status)` is true, it returns the exit status as an integer (0-255).

Bit-manipulation macros

Some bit-manipulation macros have been defined to deal with the value in the variable `status` (you need to include `<sys/wait.h>`).

`WIFSIGNALED(status)`: true for abnormal child termination

`WTERMSIG(status)`: used only when `WIFSIGNALED(status)` is true, it returns the signal number that caused the abnormal child death.

Bit-manipulation macros

Some bit-manipulation macros have been defined to deal with the value in the variable `status` (you need to include `<sys/wait.h>`).

`WCOREDUMP(status)`: true if a core file was generated.

Termination

Macro	Description
`WIFEXITED(`*status*`)`	True if status was returned for a child that terminated normally. In this case, we can execute `WEXITSTATUS(`*status*`)` to fetch the low-order 8 bits of the argument that the child passed to `exit`, `_exit`, or `_Exit`.

`WIFSIGNALED(`*status*`)`	True if status was returned for a child that terminated abnormally, by receipt of a signal that it didn't catch. In this case, we can execute
	`WTERMSIG(`*status*`)`
	to fetch the signal number that caused the termination.
	Additionally, some implementations (but not the Single UNIX Specification) define the macro
	`WCOREDUMP(`*status*`)`
	that returns true if a core file of the terminated process was generated.

`WIFSTOPPED(`*status*`)`	True if status was returned for a child that is currently stopped. In this case, we can execute `WSTOPSIG(`*status*`)` to fetch the signal number that caused the child to stop.
`WIFCONTINUED(`*status*`)`	True if status was returned for a child that has been continued after a job control stop (XSI option; `waitpid` only).

Figure 8.4 Macros to examine the termination status returned by `wait` and `waitpid`

Based on which of these four macros is true, other macros are used to obtain the exit status, signal number, and the like.

Termination

```
#include "apue.h"
#include <sys/wait.h>

void
pr_exit(int status)
{
    if (WIFEXITED(status))
        printf("normal termination, exit status = %d\n",
               WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("abnormal termination, signal number = %d%s\n",
               WTERMSIG(status),
#ifdef  WCOREDUMP
               WCOREDUMP(status) ? " (core file generated)" : "");
#else
               "");
#endif
    else if (WIFSTOPPED(status))
        printf("child stopped, signal number = %d\n",
               WSTOPSIG(status));
}
```

Figure 8.5 Print a description of the exit status

wait() and waitpid()

waitpid()

Synopsis: `pid_t waitpid(pid_t pid, int *status, int options);`

pid == −1	Waits for any child process. In this respect, `waitpid` is equivalent to `wait`.
pid > 0	Waits for the child whose process ID equals *pid*.
pid == 0	Waits for any child whose process group ID equals that of the calling process. (We discuss process groups in Section 9.4.)
pid < −1	Waits for any child whose process group ID equals the absolute value of *pid*.

wait() and waitpid() (wait_demo.c status1.c, status2.c)

waitpid()

Synopsis: `pid_t waitpid(pid_t pid, int *status, int options);`

Constant	Description
WCONTINUED	If the implementation supports job control, the status of any child specified by *pid* that has been continued after being stopped, but whose status has not yet been reported, is returned (XSI option).
WNOHANG	The waitpid function will not block if a child specified by *pid* is not immediately available. In this case, the return value is 0.
WUNTRACED	If the implementation supports job control, the status of any child specified by *pid* that has stopped, and whose status has not been reported since it has stopped, is returned. The WIFSTOPPED macro determines whether the return value corresponds to a stopped child process.

Figure 8.7 The *options* constants for waitpid

Part IV
About Core Dump

About Core Dump

Core Dump

- A core dump typically refers to a file (named `core`) containing the memory image of a particular process, or parts of it, along with other information such as the values of processor registers.
- The file is created when a program has terminated abnormally, i.e. crashed.
- It used to be important in debugging programs.

abort()

abort()

Synopsis: `void abort(void);`

The abort() function causes abnormal process termination to occur. It sends the signal `SIGABRT` to the parent process. It is declared in `stdlib.h`.

`abort()` causes a core dump (ulimit –a) ?

Example using `abort` (abort.c)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#include <sys/types.h>
#include <sys/wait.h>

int main() {
  int newpid;
  printf("before: mypid is %d\n",
  getpid());  if ((newpid = fork()) == -1 )
        perror("fork");
  else if (newpid == 0){
          printf("I am the child %d now
          sleeping...\n",getpid());  sleep(5);
          abort();
        }
        else{
          printf("I am the parent
          %d\n",getpid());  int status;
          int child_pid  = wait(&status);
          printf("My child %d has terminated\n",child_pid);

          printf("I have received the status =

          %d\n",status);  int child_status = status >> 8;
          int signal = status & 0x7F;
          int core = status & 0x80;

          printf("Child status = %d Signal = %d Core = %d\n",child_status, signal,
          core);
        }
}
```

Example using `abort()`

The output of the previous program:

```
before:   mypid is 5367
I am the parent 5367
I am the child 5368 now sleeping...
My child 5368 has terminated
I have received the status = 134  Child
status = 0 Signal = 6 Core =  128
```

Part V
Orphan and Zombie Proceses

Orphan and Zombie processes

A process that terminates does not leave the system before its parent accepts its return.

Special situations

There are 2 interesting situations:

1. A parent exits (for example, the parent has been killed prematurely) while its children are still alive. The children become orphans.
2. The parent is alive but never makes the call to `wait()`. The children become zombies.

Orphan processes

The kernel changes the `PPID` of the orphan processes to 1.
→ orphan processes are systematically adopted by the process `init` (whose `PID` is 1).
`init` accepts all its children returns.

Example of making an Orphan process

(orphan.c)

```
int main(){
 printf("Before fork\n");

 if ( fork() == 0 ){ // child
    printf("My parent is %d\n", getppid());
    sleep(6);
    printf("My parent is %d\n", getppid());
    exit(2);
 }
 // parent
 printf("had a child...\n");

 sleep(3);

 exit(1);
}
```

What is the output produced?

Zombie processes

- Zombie processes remain in the system's process table waiting for the acceptance of their return. However, they lose their resources (data, code, stack...).
- Because the system's process table has a fixed-size, too many zombie processes can require the intervention of the system administrator.

Example of making a zombie process (zombie.c)

```
int main(int argc, char *argv[]){
 int pid;

 pid = fork();
 if (pid){  // means pid !=0
   printf("I am the parent process, pid=%d\n", getpid());
   while(1)
    sleep(5);
 }
 printf("I am the child process, pid=%d\n", getpid());
 exit(0);
}
```

The program will display:

```
I am the parent process, pid=5585
I am the child process, pid=5586
```

If you look at the running processes: `ps -u your_user_id`

```
PID TTY TIME CMD
5585 pts/4 0:00 make_zom
5586 ?  0:00 <defunct>
```

Part VI
Assignment #2: Overview

Assignment #2: Overview

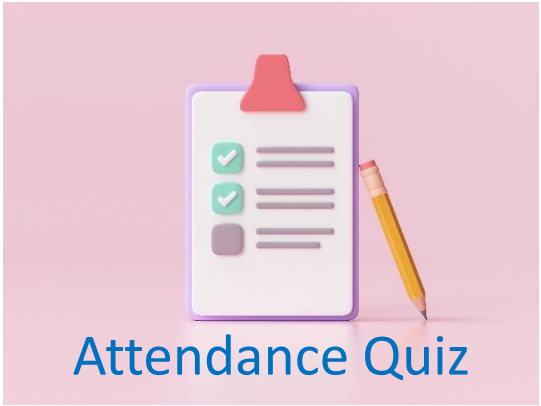
- It’s all about Coding!
- Read your examples
- Access your textbook
- Work methodology:
  - Start as early as possible
  - Read carefully!
  - Read again!
  - Reach out for clarifications early
  - Make your example easily readable
- Submit way before the deadline
- Avoid using the grace period

Assignment #2: Overview

- It's all about coding
- Questions on file I/O (continued):
  - Standard calls
  - System calls
  - Write program
  - Re-write program
- Make sure it compiles
- Make sure the logic is correct
- Submission:
  - Code
  - Video, explaining what you did

Part VII
Attendance Quiz (Not graded)

Quiz



University of Windsor

Thank you!
Questions?

Dr. Andreas S. Maniatis
Assistant Professor
Andreas.Maniatis@UWindsor.ca
http://www.linkedin.com/in/andreasmaniatis